VPN-LAN Bidirectional Exposure & Ingress Allowlist — Design

VPN-LAN Bidirectional Exposure & Ingress Allowlist — Design

Revision: 1 **Last modified:** 2026-07-01T20:15:00Z **Status:** Design reference — Phase 12 of PLAN.md §5 (operator mandate 2026-07-01: exposure MUST work fully in BOTH directions). The **ingress-allowlist logic is provable autonomously now** (teeth: ../../../tests/vpn_lan/ingress_allowlist_teeth.sh); live bidirectional round-trips are **operator-gated** (return-route on both sides, §11.4.122/§11.4.133). **Authority:** Inherits constitution/Constitution.md per §11.4.35. Companion to PLAN.md (§2 routing map [now bidirectional], §4 item 6 [ingress = mirror of egress SSRF floor], §5 Phase 12), architecture.md (§2 per-protocol routing table, §4 SSRF trust boundary), and the egress teeth ../../../tests/vpn_lan/ssrf_carveout_teeth.sh whose structure the ingress teeth mirror exactly. **Feature workstream:** feature/vpn-aware-dynamic-routing (§11.4.167)

---

0. The operator mandate (verbatim intent, 2026-07-01)

> "Exposure of network hosts and services MUST work in both directions ... Everything MUST WORK fully bi-directional without any issues, fully covered with all supported tests, tested, validated and verified with real results, no false results and no bluff of any kind!"

Direction 1 — **egress** (built): a helix_proxy-side host reaches a VPN host (L3 route over ppp0 + the scoped SSRF carve-out, PLAN.md §4, architecture.md §4). Direction 2 — **ingress** (this

mandate): a VPN host reaches an **exposed proxy-side service**. Ingress needs two new things that egress does not: **(a)** a *return route* on BOTH sides to each other's reachable subnet, and **(b)** an **INGRESS ALLOWLIST** — default-deny, only allowlisted `(proxy-side service, VPN host)` pairs permitted. The ingress allowlist is the **mirror of the egress SSRF floor** and is a **NEW attack surface**: the correct posture is never "expose everything," it is default-deny + a narrow explicit allowlist with the same anti-bluff teeth (`../../../tests/vpn_lan/ingress_allowlist_teeth.sh`) and the same operator-gating discipline.

Honest boundary up front (§11.4.6): this document + the ingress teeth prove the ingress *allowlist logic* — default-deny holds, only the exact allowlisted pair is permitted, an out-of-allowlist inbound is refused — **fully autonomously, now**. They do NOT and cannot prove a *live* bidirectional round-trip: that requires the return-route configured on BOTH the remote and proxy sides (operator-gated, §11.4.122/§11.4.133). A live bidirectional capability is "done" only when its runtime signature verifies **both directions** with captured evidence on a genuinely-up bridge (§11.4.108). Until then the live both-way paths honestly SKIP (`network_unreachable_external`, §11.4.3), never fake-PASS.

---

1. The return-route model (both sides route to each other's reachable subnet)

A Layer-3 VPN moves unicast IP packets between reachable subnets. For **any** exchange to complete, each side must have a route to the other's source/destination address — this is symmetric by construction. What differs between egress and ingress is **who initiates** and therefore **what must exist for the initiating packet to be delivered and its reply accepted**.

1.1 Cryptokey routing = the return route is not optional (WireGuard, FACT)

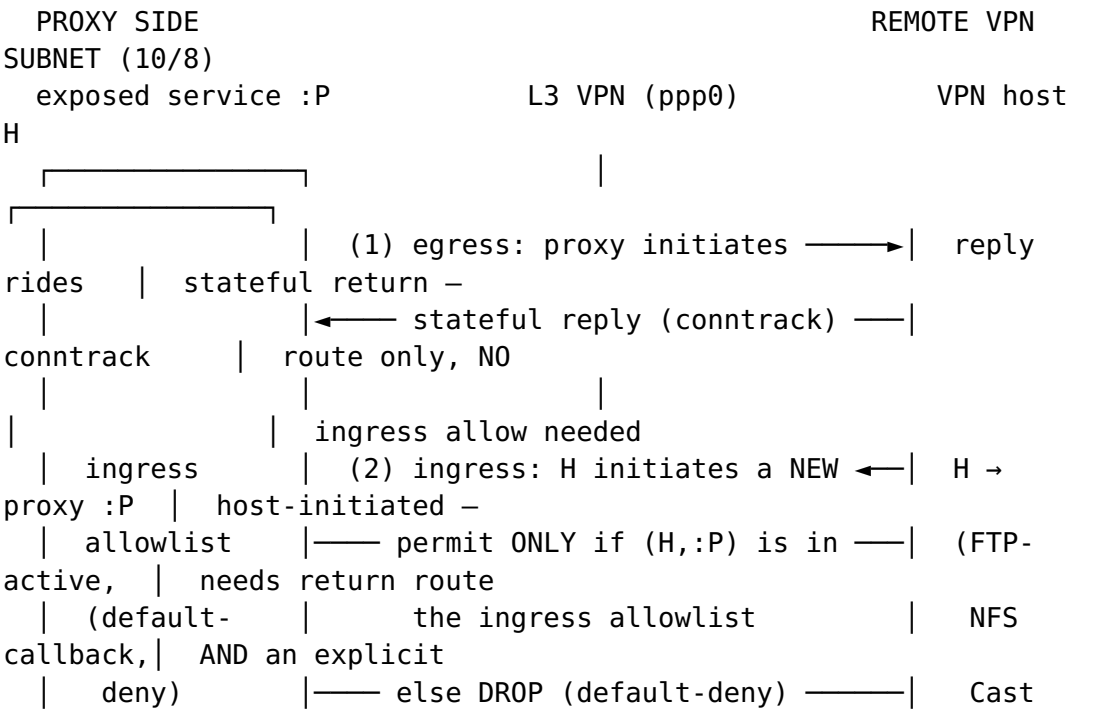
On the WireGuard leg the routing table AND the inbound access-control list are **the same field** — `AllowedIPs`. When sending, WireGuard uses `AllowedIPs` as a routing table (match the destination IP → pick the peer to encrypt for). When receiving, the **same** list is an ACL: a decrypted packet whose inner *source* IP falls outside the sending peer's `AllowedIPs` is **silently dropped, even if correctly encrypted** (FACT — WireGuard whitepaper §2 "Cryptokey Routing"; defguard "AllowedIPs Explained"). Consequence: for a VPN host to reach a proxy-side service, the **proxy-side reachable subnet must appear in the remote peer's `AllowedIPs`** (so the remote will route the inbound packet to the tunnel) AND the **remote host's address must appear in the**

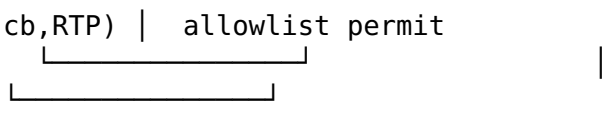
proxy-side peer's `AllowedIPs` (so the proxy-side WireGuard accepts the decrypted inbound packet). Both entries are required; a one-sided `AllowedIPs` is the single most common cause of "the tunnel is up but the reverse direction is silently dropped." On the L2TP/PPP leg the analogous requirement is a `ppp0` route on the remote side back to the proxy-side reachable subnet (INFERENCE — the PPP link carries unicast IP; the kernel needs a route entry for the reverse destination; svord owns that route, helix_proxy only *verifies* reachability, never re-routes host tables autonomously, §11.4.133).

1.2 Stateful-return vs. host-initiated (the distinction that decides ingress)

Two very different things are both loosely called "bidirectional":

- **Stateful return traffic** — the *reply* to a connection the proxy side initiated. This rides the connection-tracking state the outbound SYN created; it needs the route to exist but needs **no** inbound allowlist entry, because it is not a *new* inbound flow. All of egress (Direction 1) is this: the proxy initiates, the VPN host replies, conntrack lets the reply back. **This already works and is not the subject of the ingress allowlist.**
- **Host-initiated ingress** — a *new* connection the VPN host opens **to** a proxy-side service (FTP-active data connect, NFS lock callback, Cast status callback, `adb reverse` target, an RTP stream toward the proxy side). This is a fresh inbound flow with no prior outbound state to ride. It requires **both** the return route (§1.1) **and** an explicit ingress-allowlist permit (§3) — this is exactly the new attack surface, and default-deny is the only safe floor.





return route required on BOTH sides for either arrow to complete (§1.1)

Rule of thumb: a route makes replies possible; only an *allowlist permit* makes a **host-initiated inbound** possible — and it must be granted per exact `(service-port, VPN-host)` pair, never host-wide or port-wide, never "expose everything."

---

2. Per-protocol both-way needs (the operative table)

Which protocols genuinely need the reverse (host-initiated ingress) direction, what breaks if only one-way is provisioned, and how helix_proxy handles it. Rows marked **INFERENCE** are reasoned from the cited mechanics, not from a live capture (§11.4.6).

Protocol	Which direction(s) it initiates	What breaks if only proxy→VPN (one-way)	How helix_prox handles it
FTP passive (PASV)	Client→server for BOTH control (21) and data (pinned PASV range)	Nothing — passive is the one-way-friendly mode	**Default pa** Route 21 + the server's pinned passive ran PASV must advertise t routable `10` (Phase 3). I ingress allowlist en needed.
FTP active (PORT)	Control client→server (21); **data server→client** (server opens back to the client's advertised port, source :20)	Active-mode transfers hang/ fail — the server cannot open the return data connection to the client (FACT — slacksite; jscape)	Prefer passive. If active is required: return-rout an ingress-allowlist permit for `(FTP-server VPN-host → client's pinned acti data port`

Protocol	Which direction(s) it initiates	What breaks if only proxy→VPN (one-way)	How helix_prox handles it
			`range on th` `proxy side)` FTPS encry control so r ALG can rewrite PO — routing, proxying, is the fit.
NFS lock (NLM `lockd`)	Client→server locks (2049); **server→client async GRANT / blocking-lock callbacks**	File locking degrades — blocking locks never get their GRANT callback; advisory locking becomes unreliable (FACT — Oracle "About NFS Services"; man7 statd)	Pin `lockd` po (`--port`, bot TCP+UDP) return-rout ingress-allowlist permit for t server→clie callback po
NFS status monitor (NSM `rpc.sta` `td`)	**Both peers notify each other** — `SM_NOTIFY` on reboot (server→client AND client→server)	Reboot recovery breaks — stale locks are never reclaimed/released after either side reboots (FACT — man7 `statd(8)`: sm-notify sends SM_NOTIFY to each monitored peer; a remote reboot notifies the local rpc.statd)	Pin `rpc.sta` `--port` + `--` `outgoing-po` return-rout ingress-allowlist permit for t statd notify port in **bot** directions.
portmapper / rpcbind	Client→server discovery (111), used by NLM/NSM to find dynamic ports	RPC service discovery fails — clients cannot resolve the dynamic `lockd`/`statd` ports (FACT — Oracle: rpcbind required on both client and server, port 111)	rpcbind:11 reachable both ways; the dynami ports so the allowlist entries are stable (INFEREN — dynamic ports otherwise force a hos wide allow, defeating narrowness
Cast / DIAL callbacks	Controller→receiver control (8008/8009);	Cast degrades to fire-and-forget — the controller loses status/feedback	Return-rou + ingress-allowlist

Protocol	Which direction(s) it initiates	What breaks if only proxy→VPN (one-way)	How helix_prox handles it
	receiver→controller status callbacks	transitions the receiver pushes back (INFERENCE — CASTV2 is a bidirectional protocol; the receiver reports state changes)	permit for (Cast recei VPN-host → controller callback po on the prox side). Discovery s reflects (Ph 5); control routes (Pha 6).
mDNS / DNS-SD	Inherently bidirectional — query one way, response the other; the reflector reflects queries, responses AND probes across interfaces	Discovery is one-sided — services on the far subnet are never learned / conflicts never detected (FACT — Debian avahi-daemon.conf(5): reflector reflects to all local interfaces; deepwiki avahi reflects queries+responses+probes)	Remote-sid Avahi enabl reflector=y (Phase 5), **inherently bidirectio** not an ingress-allowlist ca (multicast o the remote segment, n host-initiat unicast into the proxy side).
SSDP / UPnP / WS-Discovery	Multicast M-SEARCH + unicast responses — bidirectional on the segment	Discovery one-sided (as mDNS)	Remote-sid SSDP/WS-D relay (Phas 5); bidirectiona on the rem segment; n a proxy-sid ingress cas
adb reverse	Device→host connect-back (an on-device app dials a host-side service)	On-device targets of adb reverse (RN Metro bundler, a local dev server) are unreachable — the reverse channel never forms (FACT — linuxcommandlibrary adb-reverse; android.googlesource system/core)	The reverse forward is **multiplexe inside the already-establishe adb connectio** over routed 5555 — it do

Protocol	Which direction(s) it initiates	What breaks if only proxy→VPN (one-way)	How helix_prox handles it
			not require separate proxy-side ingress-allowlist po (INFEREN — adb reve rides the ac transport, r a new inbo socket to th proxy host) Requires th routed 555 up + the ac server reachable.
SIP signalling + RTP media	SIP both ways (register/invite); **RTP is bidirectional UDP** (each side sends media to the other's negotiated port)	One-way audio / no media — the reverse RTP stream toward the proxy side is dropped (INFERENCE — RTP negotiates a receive port per side; the reverse leg is host-initiated inbound)	Return-rou + ingress-allowlist permit for t negotiated RTP receiv port range the proxy s note FTP/S class ALG blindness under TLS route, don't proxy-rewr
Any PORT-based / callback protocol	Establishes a control channel one way, a **data/callback channel the other way**	The callback/data channel fails while control "looks connected" (the classic one-way-provisioned bug)	Treat every such protoc as **host-initiated ingress**: pi its callback port, add th exact `(VPN-host, callback-po` ingress-allowlist pa prove both directions (§4).

The load-bearing observation: the protocols that break under one-way provisioning are exactly the ones with a **server→client / receiver→controller / device→host callback**. Each such callback is a *host-initiated inbound flow* and therefore each needs a narrow ingress allowlist permit — never a blanket "allow the VPN subnet inbound."

---

3. The INGRESS security surface = default-deny + explicit allowlist

The ingress allowlist is the **mirror** of the egress SSRF floor, but its *default posture is inverted* — and that inversion is deliberate, not incidental.

3.1 Why the posture is inverted vs. egress (design FACT)

Axis	Egress floor (`config/dante/sockd.conf`)	Ingress allowlist (this design)
Default	**Allow public** internet, with a *block-list floor* denying RFC1918 / link-local / loopback / metadata	**Deny everything inbound**, with an *allow-list* permitting only exact exposed pairs
Match order	First-match top-down: narrow subnet ALLOW above the broad internal-DENY	First-match top-down: narrow `(host,port)` ALLOW; anything unmatched falls to the implicit DENY
Unit of grant	A destination CIDR (where the proxy may reach *out*)	An exact `(proxy-service-port, VPN-host)` pair (who may reach *in*, to what)
Risk if it drifts	SSRF — the proxy reaches internal targets it should not	**Exposure** — an unallowlisted VPN host reaches a proxy-side service it should not

The inversion is because **inbound exposure is a strictly higher-risk surface than outbound reach**: an over-broad egress rule lets the proxy *talk to* something; an over-broad ingress rule lets an arbitrary remote host *open connections into* proxy-side services. So ingress is default-**deny** (allow-list), never default-allow-with-a-floor. "Expose everything" is explicitly forbidden (operator mandate + `PLAN.md` §4 item 6).

3.2 The ingress allowlist contract (what the teeth enforce)

- **Default-deny holds.** Any inbound `(VPN-host, proxy-service-port)` that is not an exact allowlist match is DENIED. There is no catch-all permit and no "allow the whole subnet."
- **Only the exact pair is permitted.** The allowlisted `(service-port, VPN-host)` is PERMITTED — and *nothing wider*.
- **Narrow, not host-wide or port-wide.** A **different host** (same port) ⇒ DENIED; a **different port** (same host) ⇒ DENIED. The permit is per exact pair.
- **Teeth, not a bluff gate.** A paired §1.1 mutation renders a golden-bad policy (allow-all / default-permit) and the teeth **FAIL** it (§11.4.107(10)): the allowlist logic that PASSes the good policy MUST reject the golden-bad, else the gate proves nothing. This is the ingress mirror of `ssrf_carveout_teeth.sh SSRF_MUT=1`.
- **Operator-gated live path.** Granting a live ingress pair changes remote-side + proxy-side config (§11.4.122 interactive-question-before-change / §11.4.133 target-safety). The *allowlist logic* is provable now against a local-stub policy (exactly as the Phase-1 egress teeth prove the carve-out logic without touching the live data-plane); the *live* grant is parked until the operator authorises it.

3.3 Autonomous proof available now — `tests/vpn_lan/ingress_allowlist_teeth.sh`

A deterministic first-match-wins ingress-allowlist evaluator (the ingress twin of the egress evaluator in `ssrf_carveout_teeth.sh`): given an inbound `(vpn-host, proxy-service-port)` it decides permit/deny against a default-deny + allowlist policy rendered into a scratch file. Normal run asserts the §3.2 contract and exits 0 with PASS lines + captured evidence under `qa-results/vpn_lan/phase12/<ts>/`. `INGRESS_MUT=1` renders the golden-bad allow-all policy, runs the same teeth, and exits **1** — the teeth caught the golden-bad, proving they are not a bluff gate. It opens **no** listening socket, edits **no** live config, runs **no** `pkill`/`kill`, and touches **no** data-plane port — pure policy logic.

---

4. All-test-type coverage plan (§11.4.169) — bidirectionality across every test type

Every protocol's both-directions behaviour is exercised across the full §11.4.169 test-type set. The **autonomous-now** column is the security-critical ingress-allowlist logic + parsers that need no live

bridge; the **operator-gated** column is the live round-trip that honestly SKIPs (`network_unreachable_external`, §11.4.3) until the bridge + return-route are up, and emits real captured evidence (§11.4.5/§11.4.69) when they are.

Test type (§11.4.169)	Bidirectional assertion	Autonomous now?	Evidence / honest-SKIP
unit	Ingress-allowlist evaluator: default-deny, exact-pair permit, host/port narrowness; policy parser edge cases	YES	`ingress_allowlist_teeth.sh` evidence dir; golden-good/golden-bad self-validation
integration	Each protocol driven against the real service **both ways** (e.g. NFS lock held from proxy side + `SM_NOTIFY` observed inbound) via the bridge	Live-gated	Real capture when up; SKIP when bridge/service down
e2e	Full round-trip proving BOTH directions completed (FTP-active transfer; Cast status callback received; `adb reverse` target reached)	Live-gated	Round-trip evidence (bytes/sha256/status transition) or honest SKIP
full-automation	Re-runnable, no manual step (§11.4.98); both-way assertions run headless	Logic: YES · live: gated	Deterministic across N iters (§11.4.50); SKIP when down
security	Ingress default-deny	YES	`INGRESS_MUT=1 ⇒ rc=1`; egress

Test type (§11.4.169)	Bidirectional assertion	Autonomous now?	Evidence / honest-SKIP
	holds; an out-of-allowlist inbound `(host,port)` is REFUSED (the teeth + `INGRESS_MUT=1` mutation); egress SSRF floor still GREEN (no ingress rule weakens it)		`ssrf_carveout_teeth.sh` unchanged/GREEN
stress + chaos (§11.4.85)	Callback storms (many concurrent NFS GRANT/ SM_NOTIFY); reverse-channel drop + reconnect; boundary ports (0, 65535, off-by-one around the pinned range)	Logic: YES · live: gated	Per-iteration latency + categorised-recovery evidence; SKIP live when down
DDoS / load-flood	Inbound-flood against the exposed pair: default-deny refuses the flood cleanly (no fail-open); the allowlisted pair degrades gracefully	Logic: YES · live: gated	Refusal-count + throughput evidence
concurrency / atomicity	Simultaneous both-direction flows do not cross-contaminate (single-resource-owner per device, §11.4.119)	Live-gated	Non-interleaved evidence per direction

Test type (§11.4.169)	Bidirectional assertion	Autonomous now?	Evidence / honest-SKIP
race / deadlock	No lock-order inversion when both directions are provisioned; ingress evaluator has no shared-lock blocking	YES (evaluator)	`sh -n`/`bash -n` clean; deterministic verdict
memory	Long-soak both-way session shows no unbounded growth (evaluator + harness)	YES (harness)	RSS census over soak
benchmarking	Ingress-evaluate latency p50/p95/p99 vs. baseline; round-trip latency both ways	Logic: YES · live: gated	Latency distribution artifact
Challenges (challenges submodule)	A Challenge entry per protocol scoring PASS only on positive both-direction captured evidence (§11.4.69)	Logic: YES · live: gated	Challenge `result.json` with evidence path; SKIP when down
HelixQA	A bank case per protocol driven by an autonomous session asserting BOTH directions	Logic: YES · live: gated	HelixQA wire evidence; honest SKIP when down

Bridge-down ⇒ honest SKIP across the board (§11.4.3) — **never** a fake PASS. A live both-way capability is only "done" when its runtime signature verifies **both** directions with captured evidence on a genuinely-up bridge (§11.4.108).

5. Honest boundary (§11.4.6)

- **Provable autonomously, now:** the ingress-allowlist *logic* — default-deny holds, only the exact `(service-port, VPN-host)` pair is permitted, a different host or port is denied, and the teeth reject a golden-bad allow-all policy (`INGRESS_MUT=1 ⇒ rc=1`). This is the mirror of the Phase-1 egress SSRF teeth and needs no live VPN.
- **Requires the operator (parked, §11.4.21):** every *live* bidirectional round-trip. It needs the return route configured on BOTH the remote and proxy sides (WireGuard `AllowedIPs` on both peers, §1.1 / a `ppp0` reverse route the svord bridge owns), the pinned callback ports on the far service, and an explicit ingress-allowlist grant — all of which change remote-side and/or proxy-side config and are therefore §11.4.122/§11.4.133 operator-gated. Until authorised, the live both-way paths honestly SKIP (§11.4.3), never fake-PASS.
- **Not claimed:** this document does NOT claim any live bidirectional protocol works today. It designs the model, enumerates the per-protocol both-way needs, defines the ingress security surface, and ships the autonomous ingress-allowlist teeth. Each live capability becomes "done" only under §11.4.108 (runtime signature verified BOTH directions on a genuinely-up bridge) + independent review (§11.4.142) + the §11.4.169 test-type matrix.
- **`adb reverse` and multicast are reasoned, not captured (INFERENCE):** `adb reverse` is taken to ride the existing adb transport (no separate proxy-side ingress port) from the adb protocol's documented behaviour; mDNS/SSDP bidirectionality is a remote-segment reflector property, not a proxy-side ingress case. Both are marked INFERENCE and are confirmed only by live capture when the bridge is up.

Sources verified 2026-07-01

- WireGuard cryptokey routing / `AllowedIPs` bidirectional (routing-table-on-send, ACL-on-receive; both peers need complementary `AllowedIPs`): https://www.wireguard.com/papers/wireguard.pdf (whitepaper §2 "Cryptokey Routing"); https://defguard.net/blog/allowedips-explained/; https://github.com/pirate/wireguard-docs.
- FTP active vs. passive (active = server initiates the data connection back to the client, source port 20; passive = client initiates both): https://slacksite.com/other/ftp.html; https://www.jscape.com/blog/active-v-s-passive-ftp-simplified.
- NFS NLM (`lockd`) / NSM (`rpc.statd`) callbacks + rpcbind, and their bidirectional `SM_NOTIFY` / GRANT requirement + port-pinning (`--port` / `--outgoing-port`, `lockd` port): https://man7.org/linux/man-pages/man8/statd.8.html; https://docs.oracle.com/

en/operating-systems/oracle-linux/8/nfs/about-nfs-services_concept.html.

- mDNS reflector inherent bidirectionality (Avahi `enable-reflector=yes` reflects queries, responses and probes across all local interfaces): https://manpages.debian.org/unstable/avahi-daemon/avahi-daemon.conf.5.en.html; https://deepwiki.com/avahi/avahi/2.6-mdns-reflector.
- `adb reverse` device→host connect-back (reverse of `adb forward`; rides the adb transport; requires the adb server to maintain the connection): https://linuxcommandlibrary.com/man/adb-reverse; https://android.googlesource.com/platform/system/core/+/252586941934d23073a8d167ec240b221062505f.

Deep-research pass per §11.4.150 (multi-angle: WireGuard routing / FTP modes / NFS callback mechanics / mDNS reflection / adb reverse). Access date 2026-07-01. Latest-source verified per §11.4.99. No claim rests on training memory; each mechanic above is cited to an authoritative source and cross-checked against a second where the claim is load-bearing.